



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير متطلبات أداء نظام التجارة الإلكترونية

نصوح الدهيم 43371

زين محمد الشيخ 43150

تيسير مطر 43082

احمد الحريري 43014

الحمزة مطر 43034

بإشراف: م. حذيفة عقيل

تم العمل على اطار العمل Django

1. الوصول المتزامن وسلامة البيانات (Concurrent Access & Data Integrity)

المشكلة:

عندما يقوم عدة مستخدمين بمحاولة شراء نفس المنتج في نفس اللحظة، تحدث مشكلة السباق (Race Condition) قبل الحل كانت الطلبات تقرأ كمية المخزون المتاحة وتحديثها في وقت واحد مما أدى إلى خصم الكمية بشكل غير صحيح وبيع منتجات بكميات تتجاوز المخزون الحقيقي.

الحل المطبق:

- تم حل المشكلة بالاعتماد على ميزة قفل قاعدة البيانات (Database Locking) باستخدام (transaction.atomic) تم استخدام الدالة (select_for_update) عند قراءة ال Cart و Product و معلومات المستخدم مما يضمن قفل السجلات Row-Level Locking أثناء قراءتها ، بحيث يتم وضع أي طلب آخر لنفس المنتج في حالة حتى لا يحدث ضياع في البيانات أو تعارض في القيم.

- صورة لكمية المنتج قبل الطلب من قبل 10 مستخدمين في نفس الوقت

id	name	description	price	stock_quantity	image_url
1	Premium Phone 1	Reliable phone for everyday e-commerce testing and catalog browsing.	187.48	100	https://picsum.photos/seed/product

صورة ل معلومات المستخدمين

id	email	username	hashed_password	balance	created_at
1	user1@example.com	user1	seeded_password_hash	50000000.00	2026-05-10 21:47:20
2	user2@example.com	user2	seeded_password_hash	50000000.00	2026-05-10 21:47:20
3	user3@example.com	user3	seeded_password_hash	50000000.00	2026-05-10 21:47:20
4	user4@example.com	user4	seeded_password_hash	50000000.00	2026-05-10 21:47:20
5	user5@example.com	user5	seeded_password_hash	50000000.00	2026-05-10 21:47:20
6	user6@example.com	user6	seeded_password_hash	49992500.00	2026-05-10 21:47:20
7	user7@example.com	user7	seeded_password_hash	50000000.00	2026-05-10 21:47:20
8	user8@example.com	user8	seeded_password_hash	50000000.00	2026-05-10 21:47:20
9	user9@example.com	user9	seeded_password_hash	50000000.00	2026-05-10 21:47:20
10	user10@example.com	user10	seeded_password_hash	50000000.00	2026-05-10 21:47:20

حيث ال Cart لكل واحد منهم تحوي على واحد من ال Product 1 و عند الطلب :

Label	# Sa...	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/s...	Sent KB/sec	Avg. Bytes
High Concurrency Shoppers:Submit Checkout Order	10	2109	2088	2145	16.65	0.00%	3.3/sec	1.33	0.76	411.1
TOTAL	10	2109	2088	2145	16.65	0.00%	3.3/sec	1.33	0.76	411.1

و لكن حدث أخطاء أثناء عمليات الشراء حيث أصبحت الكمية المتوفرة 93 قطعة بدل 90 قطعة

بعد الحل :

يتم تحديث المخزون بشكل دقيق ومنع أخطاء الكميات المتوفر حتى لو كان المستخدمون متزامنين.

بعد إعادة نفس الاختبار

Label	# Sa...	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/s...	Sent KB/sec	Avg. Bytes
High Concurrency Shoppers:Submit Checkout Order	10	2101	2047	2140	29.15	0.00%	3.4/sec	1.35	0.76	411.1
TOTAL	10	2101	2047	2140	29.15	0.00%	3.4/sec	1.35	0.76	411.1

id	name	description	price	stock_quantity	image_url
1	Premium Phone 1	Reliable phone for everyday e-commerce testing and catalog browsing.	187.48	90	https://picsum.photos/seed/product

تم تحديث الكمية بشكل صحيح

2. إدارة الموارد والتحكم في السعة (Resource Management & Capacity Control)

المشكلة:

استقبال عدد كبير من الطلبات المتوازية يتسبب في استهلاك مفرط لموارد النظام مما يؤدي إلى البطء أو الانهيار ونسبة خطأ مرتفعة بدون استغلال الموارد بالشكل الأمثل

الحل المطبق:

تم بناء مدير موارد مخصص ResourceManager مدعوم ب ResourceManagerMiddleware

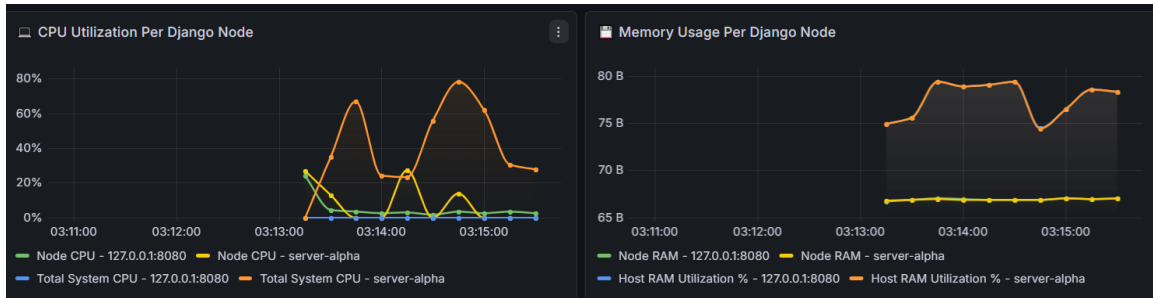
تم تعريف بوابتين: الأولى Workers وتسمح بعدد محدد من الطلبات بالعمل بالتوازي MAX_WORKERS=20 ، والثانية Admission وتسمح بعدد محدد من الطلبات بالانتظار في الطابور MAX_QUEUE_SIZE=100. أي طلب يتجاوز هذه السعة يتم رفضه بالإضافة إلى تطبيق Cached Thread Pool و ذلك لايقاف ال Threads في حال كانت لا تعمل لتقليل استخدام الموارد بعد مرور زمن محدد من حالة ال Idle

بعد الحل:

المخرجات: استقرار النظام تحت الضغط وإرجاع استجابة (503 Service Unavailable) للطلبات الفائضة

صورة لاختبار قبل الحل

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
High Concurr...	328	48	4	220	25.33	67.68%	84.0/sec	34.19	14.71	417.0
High Concurr...	323	50	4	149	20.75	63.16%	82.9/sec	797.05	14.32	9842.2
TOTAL	651	49	4	220	23.19	65.44%	166.6/sec	828.79	28.98	5093.4



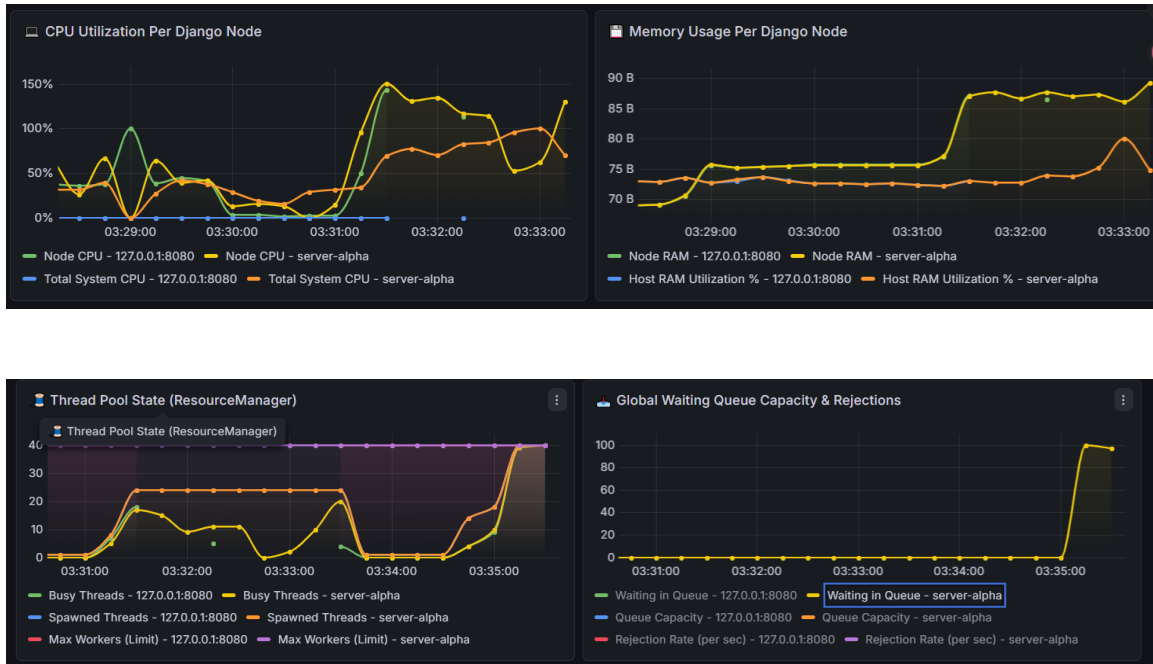
لا يتم استغلال الموارد بالشكل الأمثل مع حصول أخطاء في ال Requests

الصورة بعد تطبيق Thread pool و ResourceManager

اعادة الاختبار

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
High Concurr...	281	33	8	106	18.26	0.00%	89.4/sec	36.27	15.90	415.6
High Concurr...	277	41	13	118	20.09	0.00%	88.0/sec	2226.21	15.73	25899.0
TOTAL	558	37	8	118	19.60	0.00%	176.5/sec	2251.72	31.47	13066.0

ارتفع ال Throughput وانخفض معدل الخطء



3. المعالجة غير المتزامنة (Asynchronous Queues)

المشكلة:

المهام الثقيلة مثل عملية الـ Checkout وإرسال الإشعارات Notifications كانت تتم داخل مسار الطلب الأساسي، مما يجبر المستخدم على الانتظار لفترة طويلة حتى يتلقى الاستجابة وينهي الطلب.

الحل المطبق:

تم استخدام Async Queues لتنفيذ هذه المهام بعيداً عن مسار الطلب الرئيسي حيث تم تطبيق نظام طابور مهام داخلي CheckoutQueue باستخدام Redis و Celery workers و استدعاء دالة إرسال الإشعارات لتعمل كمهام خلفية بعد اكتمال الشراء أي عملية الشراء ثلاث مسارات تعمل بشكل Async و هي:

1: مسار يقوم ب تسجيل الطلب

2: مسار check out and send notification

3: مسار يعيد للمستخدم استجابة

بعد الحل:

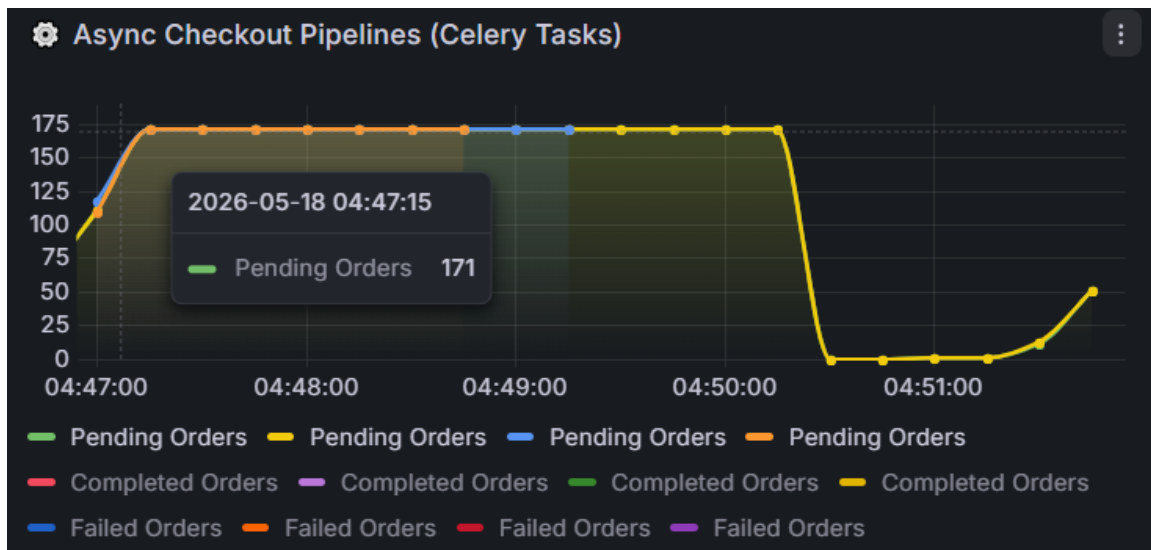
- استجابة سريعة للطلب، وتتم المعالجة الثقيلة والإشعارات في الخلفية مع إعادة المحاولة التلقائية عند فشل المهمة.
قبل المعالجة : اخذ الطلب حوالي 6 ثواني

Sampler result		Request	Response data
Thread Name	High Concurrency Shoppers 1-1		1 POST http://127.0.0.1:8080/orders/checkout
Sample Start	2026-05-18 04:37:19 AST		2
Load time	6368		3 POST data:
Connect Time	1		4 {
Latency	6368		5 "user_id": 1
Size in bytes	411		6 }
Sent bytes	231		7
Header size in bytes	200		8 [no cookies]
			9

بعد المعالجة : الطب اخذ حواي ثانيتين فقط

Sampler result		Request	Response data
Thread Name	High Concurrency Shoppers 1-1		1 POST http://127.0.0.1:8080/orders/checkout
Sample Start	2026-05-18 04:36:20 AST		2
Load time	2102		3 POST data:
Connect Time	1		4 {
Latency	2102		5 "user_id": 1
Size in bytes	411		6 }
Sent bytes	231		7
Header size in bytes	200		8 [no cookies]
			9

صورة توضح Queue الدفع :



4.المعالجة البيانات الضخمة على دفعات(Batch Processing)

المشكلة:

تنفيذ عملية الجرد اليومي للمبيعات دفعة واحدة يؤدي إلى سحب بيانات ضخمة مما يؤدي إلى استهلاك عالي جداً للموارد وإبطاء النظام بأكمله خلال فترة الجرد و مع احتمالية تزامن هذه العملية مع فترات الذروة على الخادم مما يسبب بطا شديد و مشكلة معالجة البيانات كل سجل بشكل متتابع و ليس على التوازي.

الحل المطبق:

تم تجهيز Background Job لمعالجة البيانات على دفعات Chunks من خلال تقسيم عملية جرد السجلات إلى حزم متتالية Batches يتم تقليل الضغط على قاعدة البيانات و الموارد، وجعل معالجة القوائم والمخزون تتم بشكل ومستقر و سريع و بوقت يكون فيه الضغط على الخادم أقل ما يمكن خلال اليوم و يتم تحديد عدد الدفعات و حجمها بشكل تلقائي Dynamic

Partitioning و ذلك بحسب عدد ال Workers المتاحة وقت بدا العملية مع إمكانية تتبع كل Chunk و التأكد أنه اكتمل و اعادته في حال الفشل دون عطب البيانات و توليد تقرير PDF.

بعد الحل:

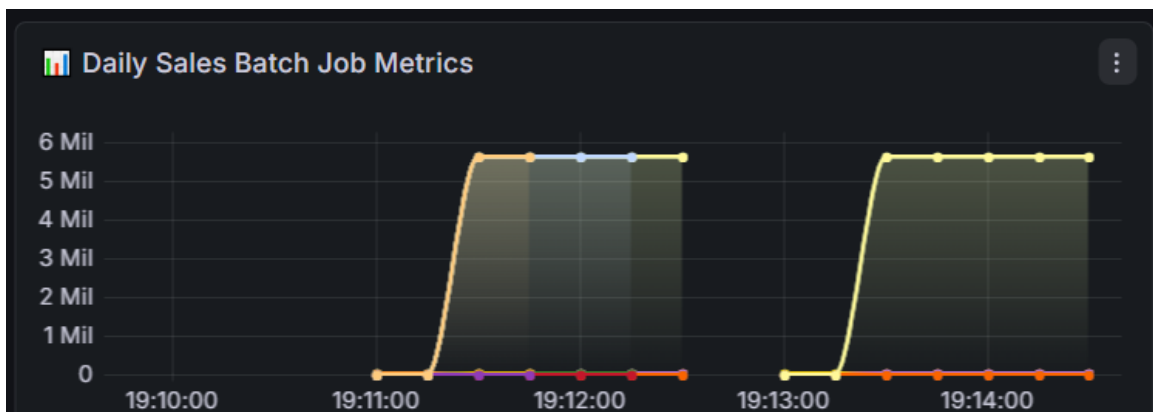
إنجاز عملية الجرد بكفاءة وسرعة، مع بقاء استهلاك ال RAM وال CPU عند مستويات منخفضة.

صورة للوقت قبل التنفيذ كان 3501 ثانية:

id	date	status	processed_orders	total_revenue	total_execution_time
30	2026-05-17	completed	20939	5640878.39	3501

بعد التنفيذ التحسينات أصبح 32 ثانية .

id	date	status	processed_orders	total_revenue	total_execution_time
34	2026-05-17	completed	20939	5640878.39	32



جدول يوضح ال Chunks عددها 20 يساوي عدد ال Workers في حالة خمول

id	chunk_index ↑	status	total_revenue	processed_count	error_message
507	1	completed	187480.00	1000	NULL
508	2	completed	187480.00	1000	NULL
509	3	completed	187480.00	1000	NULL
510	4	completed	187480.00	1000	NULL
511	5	completed	187480.00	1000	NULL
512	6	completed	187480.00	1000	NULL
513	7	completed	187480.00	1000	NULL
514	8	completed	187480.00	1000	NULL
515	9	completed	187480.00	1000	NULL
516	10	completed	187480.00	1000	NULL
517	11	completed	187480.00	1000	NULL
518	12	completed	187480.00	1000	NULL
519	13	completed	187480.00	1000	NULL
520	14	completed	187480.00	1000	NULL
521	15	completed	187480.00	1000	NULL
522	16	completed	187480.00	1000	NULL
523	17	completed	187480.00	1000	NULL
524	18	completed	187480.00	1000	NULL
525	19	completed	187480.00	1000	NULL
526	20	completed	183917.88	939	NULL

5. توزيع الأحمال (Load Distribution)

المشكلة:

الاعتماد على Server واحد يجعل النظام عرضة للانحيار الكامل في حالة تعطل هذا الخادم، ويحد من القدرة على التوسع لاستيعاب الكثير من المستخدمين.

الحل المطبق:

يتم إعداد خادم موزع أحمال (Load Balancer) لتوزيع طلبات Requests استراتيجياً التوزيع المستخدمة والمحاكاة هي Weighted Round Robin والتي توزع الطلبات على جميع الخوادم المتاحة كل واحد على حسب موارده بالاعتماد على الوزن.

تم انشاء ثلاث سيرفرات يقوم Ngnix بتوزيع الطلبات عليهم و يتم إيقافها وتشغيلها بشكل تلقائي على حسب الأحمال

بعد الحل:

اصبح النظام قادر على التوسع الأفقي بإضافة خوادم جديدة حتى لو تعطل أحد الخوادم و معالجة عدد أكبر من الطلبات و تحقيق توزيع متساوٍ للحمل يقلل أوقات الاستجابة .

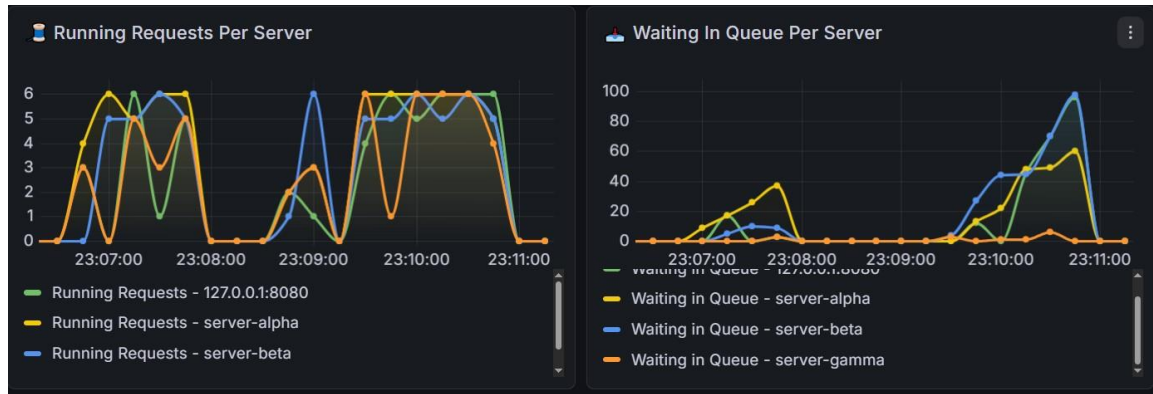
قبل عدد ال requests بالثانية حوالي 75:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
High Concurr...	591	399	46	2231	459.17	1.35%	43.9/sec	17.78	7.82	414.2
High Concurr...	527	460	69	2485	426.00	0.95%	39.4/sec	987.43	7.04	25656.3
High Concurr...	57	2910	2246	4106	478.22	0.00%	3.7/sec	1.49	0.85	411.0
TOTAL	1175	548	46	4106	695.52	1.11%	75.4/sec	864.47	13.63	11735.4

بعد التحسين أصبح حوالي 181:

Label	# Sampl...	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/s...	Sent KB/s...	Avg. Bytes
High Concurrency Shoppers: Get User Info (GET)	1525	923	19	7622	991.60	0.00%	92.6/sec	38.25	16.70	422.9
High Concurrency Shoppers: Browse Catalog (GET)	1342	863	27	7469	942.84	0.07%	81.7/sec	2063.87	14.59	25878.9
High Concurrency Shoppers: Submit Checkout Order (POST)	123	3369	2329	5820	868.04	0.00%	7.5/sec	3.05	1.73	413.6
TOTAL	2990	997	19	7622	1083.43	0.03%	181.6/sec	2100.87	32.98	11847.9

صورة توضح موارد ال Servers الثلاثة :



6. التخزين المؤقت (Caching Strategy)

تم دمج خادم Redis كطبقة تخزين مؤقت (Cache Layer) لتخزين المنتجات الأكثر طلباً، يخفف الضغط ويقلل الاستعلامات المباشرة على قاعدة البيانات تم الاعتماد على مكتبة django-redis

قبل التنفيذ: الطلبات تأخذ وقت بسبب استعلامات ال DB المتكررة

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
2. View Product ...	3136	8	1	64	6.25	0.26%	71.7/sec	39.25	12.99	560.3
Get Products	965	8	2	50	5.74	0.21%	22.1/sec	61.94	4.29	2871.4
Add Product to ...	482	46	31	117	13.55	0.00%	11.1/sec	391.45	2.87	36232.8
5. Add New Pro...	43	18	8	54	8.18	0.00%	1.0/sec	0.47	0.32	477.0
Checkout Order	187	1530	19	6281	2089.02	0.00%	4.3/sec	1.74	0.97	413.6
Daily Sales Repo...	5	22	13	39	8.80	0.00%	9.4/min	0.08	0.03	505.8
TOTAL	4818	71	1	6281	505.45	0.21%	110.2/sec	493.51	21.44	4585.4

بعد التنفيذ: تم تخزين نتيجة الاستعلام في ال Cache

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
View Product De...	3030	164	7	1351	166.09	0.00%	235.2/sec	127.53	42.71	555.1
Get Products	937	157	7	894	156.49	0.00%	72.8/sec	204.30	14.16	2872.4
Add Product to ...	450	748	39	1864	377.42	0.00%	35.2/sec	1246.29	9.14	36232.8
Checkout Order	181	715	31	1588	348.69	0.00%	14.3/sec	5.77	3.23	414.0
Add New Produ...	42	579	72	1079	313.16	0.00%	3.4/sec	1.59	1.07	477.0
Daily Sales Repo...	5	600	303	896	222.91	0.00%	28.0/min	0.23	0.10	506.0
TOTAL	4645	245	7	1864	287.15	0.00%	360.6/sec	1575.22	70.21	4472.7

7. القفل (distributed lock)

المشكلة قبل الحل: (Race Conditions) في بنية الأنظمة الموزعة (Distributed Systems) ، عند تفعيل مهام (مثل حساب أرباح المبيعات اليومية) استجابةً لطلب مستخدم أو وظيفة مجدولة (Cron) ، كان يحتمل أن تقوم عدة خوادم بتنفيذ نفس المهمة في ذات الملي ثانية. هذا أدى إلى تكرار العمليات، وإحداث فوضى في بيانات قاعدة البيانات لغياب آلية مركزية تمنع التكرار.

بعد الحل استخدام Redis Distributed Lock: تم تطبيق نمط باستخدام Redis كطبقة وسيطة ، عند بدء أي مهمة حرجية، الحصول على قفل وحيد (Lock) من Redis الخادم الذي يحصل على القفل أولاً ينفذ المهمة، بينما يتم رفض طلبات الخوادم الأخرى فوراً منعت التكرار المزدوج

8. سلامة المعاملات (Transaction Integrity - ACID)

- Atomicity

تم تغليف العمليات المركبة (الدفع + تحديث المخزون + إنشاء الطلب) داخل معاملة قاعدة بيانات واحدة باستخدام transaction.atomic لضمان

- Consistency

(Consistency) استخدام أقفال select_for_update لضمان دقة خصم المخزون وأرصدة المستخدمين أثناء عمليات الشراء المتزامنة، مما يمنع نهائياً بيع منتج غير متوفر أو وصول رصيد المستخدم للسالب .

- Isolation

(Isolation) تغليف كود الدفع وخصم المخزون بالكامل داخل transaction لضمان عمل كل طلب شراء في بحيث لا تتداخل آلاف الطلبات المتزامنة مع بعضها .

- Durability

(Durability) ضمان حفظ جميع بيانات الطلبات، والأرصدة المخصومة، والمبيعات بشكل دائم على القرص الصلب لقاعدة بيانات(MySQL) ، واستخدام جداول (Dead-Letter) لتخزين الطلبات الفاشلة، مما يضمن عدم ضياع بيانات حتى لو انهار السيرفر أو انقطعت الكهرباء.

9. اختبار الضغط(Stress Testing)

تم تجهيز النظام للعمل خلف موزع أحمال Nginx مع سكربت توسع تلقائي تم اختبار النظام باستخدام JMeter لمحاكاة 100مستخدم متزامن.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
View Product De...	3030	164	7	1351	166.09	0.00%	235.2/sec	127.53	42.71	555.1
Get Products	937	157	7	894	156.49	0.00%	72.8/sec	204.30	14.16	2872.4
Add Product to ...	450	748	39	1864	377.42	0.00%	35.2/sec	1246.29	9.14	36232.8
Checkout Order	181	715	31	1588	348.69	0.00%	14.3/sec	5.77	3.23	414.0
Add New Produ...	42	579	72	1079	313.16	0.00%	3.4/sec	1.59	1.07	477.0
Daily Sales Repo...	5	600	303	896	222.91	0.00%	28.0/min	0.23	0.10	506.0
TOTAL	4645	245	7	1864	287.15	0.00%	360.6/sec	1575.22	70.21	4472.7

10. قياس الأداء وتحليل عنق الزجاجة (Benchmarking & Bottleneck Analysis)

تم تحليل النظام باستخدام لوحة تحكم Grafana المتصلة بـ Prometheus لقياس أوقات الاستجابة ومراقبة الأداء للموارد. تم تحديد عنق الزجاجة الرئيسي في استنفاد خيوط المعالجة (Thread Pool Exhaustion) والضغط على قاعدة البيانات أثناء عمليات الدفع المتزامنة.

